ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Sensory-Motor Systems Lab
Prof. Dr. Robert Riener

Semester Thesis

Connecting AI Agents and Controllers for Instructing Rehabilitation Robots

A robot-agnostic control layer built on the Model Context Protocol

Autumn Term 2026

Supervised by:
Max-Ole von Waldow

Author:
Celal Türkmen

Contents

1 Introduction 1
2 State of the Art 2
3 The Framework 2
3.1 Design Goals 2
3.2 The Device 2
3.3 Architecture 3
4 Exercises 6
4.1 Exercise Definition 6
4.2 Reach Exercise 6
4.3 Haptic Rendering 6
5 The Demonstrator 7
6 Experiments 8
6.1 Safety 8
6.2 Model Interpretation 9
7 Conclusion, Limitations and Future Work 10
References 10
A Robot Specification 12

Abstract. Therapists operating rehabilitation robots often have to express their intentions in the technical vocabulary of the robot controller. This report presents a modular framework that uses a language model to translate natural-language instructions into robot commands while keeping safety-critical validation independent of the model. The system consists of a language-model host, a robot-agnostic core exposed through the Model Context Protocol (MCP), and a device-specific backend communicating with the hardware. Each device is described in one specification file listing its parameters with units, bounds, and rate limits, and backends are discovered at run time. Every command passes through a validator that checks types, bounds, and rate limits against the state the device reports, and rejects the whole command if any part is invalid. The framework runs on a Novint Falcon, a desktop haptic device with three degrees of freedom, under ROS 2, with two exercises and a demonstrator program. Of 20,000 generated commands, nothing invalid reached the device. A local model of four billion parameters carried out 57% of 42 therapist instructions without reasoning and 76% with reasoning enabled.

1 Introduction

Rehabilitation robots provide movement training to patients recovering from stroke and other neurological injuries. Recovery of arm function depends on how much a patient practices [1], and a machine can provide more guided repetitions in a session than a therapist can by hand. Robot-assisted arm training after stroke improves activities of daily living and arm function [2]. However, the way these devices are operated has not kept up with the pace of the advancements in AI and language models.

A rehabilitation robot takes its behavior from the numerical parameters of its controller. The parameters available for adjustment depend on the robot and its control strategy; therefore, therapists must identify the relevant parameter and choose a value for each patient, which remains one of the major challenges they face in daily practice [3].

With the emergence of language models, an alternative interface to interact with these systems has become possible. A model can be handed a set of typed functions and decide which one to call and with what arguments, rather than only producing text. A demonstrator of this idea in our own lab, Virtual Technician, found that models of four billion parameters and above can follow abstract instructions to adjust the parameters of rehabilitation robots [4].

The model can be connected to the robot in many ways, and the choice is a design decision. Tools can be described in one model provider's own calling format, which ties the device interface to that provider. In late 2024, the Model Context Protocol (MCP) standardized how models discover the tools available to an application and communicate with those tools [5, 6]. This fits well into a rehabilitation device. Its command surface is small, typed, and known in advance, a handful of parameters with units and bounds, so it maps onto a tool description without loss. With a standardized protocol, the device side does not depend on one model provider, and an interface written once can be reused by other groups. The tool surface is generated from each device's own specification file and discovered at run time, so adding a robot changes nothing on the model side. Researchers and therapists can likewise change the model without touching the robot.

This project introduces a framework that lets a therapist instruct a rehabilitation robot in ordinary language, with a deterministic check between the model and the device that the model can neither influence nor bypass.

Section 2 reviews two related works and the gap this project fills. Section 3 describes the framework: its design goals, the device it runs on, and the architecture that separates the model from the robot. Section 4 covers exercises, both the contract in the core and the two the Falcon provides. Section 5 presents the demonstrator program. Section 6 reports what was measured, on the safety of the checker and on how well the model interprets what a therapist says. Section 7 concludes with the limitations and what should be done next.

2 State of the Art

A recent review describes how AI has entered rehabilitation robotics. It reports better motor function, faster recovery, and improved overall well-being compared with conventional rehabilitation, and names cost and legal requirements as obstacles to deployment [7]. It also calls for unified protocols. The review means clinical protocols, but the same gap exists in software: each device must be integrated on its own terms, and an interface built for one device cannot easily be reused with another.

RACAS shows that a single system can control structurally different robots without retraining [8]. It provides a strong demonstration that robot-agnostic language control is achievable, acting as a bridge between high-level AI reasoning and low-level robot execution. However, the framework does not use MCP, which limits its use with standardized tool interfaces. Moreover, it was not evaluated on rehabilitation robots, so it does not address the patient-safety requirements and domain-specific constraints that a rehabilitation-focused framework must consider.

This project follows a similar agentic, tool-calling approach to RACAS, but uses a standardized protocol to separate the model and robot sides. Each robot describes itself in one specification file and one backend script, so an integration can be reused across devices. It also adds a safety layer: every command is checked against the robot's limits and current state before execution. This check runs outside the model, in a separate process the model cannot access.

3 The Framework

3.1 Design Goals

Four goals shaped the design (Table 1). *Scalability* means adding a second robot or a different model without modifying the shared code. *Safety* means that no command reaches the hardware without first being checked against the device's stated limits. *Demonstration* means testing on a real device rather than in simulation. *Privacy* means supporting a model that runs locally in the clinic, so sensitive data never leaves trusted parties.

Table 1: The four goals of the system, and the methods used to test them. Section 6 reports the results.

Goal	**What it requires**	**How it is tested**
Scalability	A second device, and a different model, without editing shared code	The model and the device code live in separate repositories that do not import each other; a device is added by declaration
Safety	No command reaches the hardware unchecked	Declared bounds, a per-parameter limit on how far one command may move a value, and a latching stop, fuzzed against the real validator
Demonstration	It runs on a real device, not in simulation	A Novint Falcon under ROS 2
Privacy	Patient data does not leave the clinic	The whole system driven by an open model running locally

3.2 The Device

The framework is demonstrated on a Novint Falcon, a desktop haptic device [9]. Three motors move its handle, which can be pushed in three directions but not turned, inside a workspace of about 10 cm in each direction (Figure 1). It can apply about 9 N of force, and this system clamps every commanded force at 8 N.

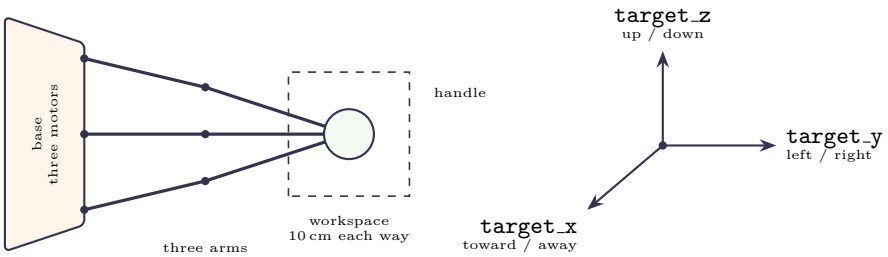


Figure 1: Schematic of the Novint Falcon, drawn after [9]. Three motors move the handle in three directions. On the right, the three axes as the device specifies them.

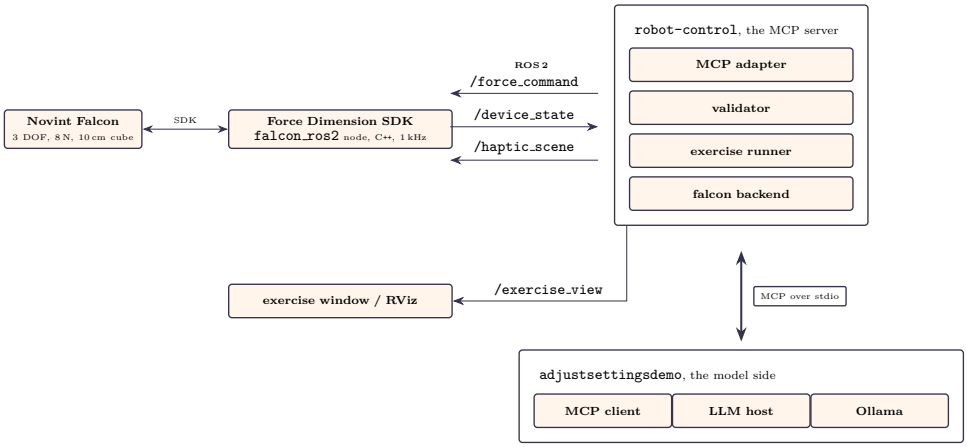


Figure 2: The high-level architecture.

The Falcon does not drive the arm along a path. It pulls the handle towards a point the therapist has set, more or less firmly, and the hand feels that pull. What the therapist adjusts is where the point is, how firmly the device pulls towards it, and what objects the hand can feel. It is also small, cheap, and safe enough to sit on a desk, which is what makes it a reasonable stand-in while the architecture is being tested, and not a clinical device.

3.3 Architecture

Figure 2 shows the high-level architecture. A C++ node holds the Force Dimension SDK [10] and closes the force loop at 1 kHz. It exchanges commands and state with the server over ROS 2 [11]. The server publishes the tool surface, validates, dispatches, audits, and runs exercises. The model side is a separate process that reaches the server only over MCP.

Three Components

The code is split into three repositories (Figure 3): `adjustsettingsdemo`, the model host;[1] `robot-control`, the robot-agnostic core;[2] and `falcon`, the device code.[3] This split makes the system more extensible, scalable, and maintainable. Each further robot adds one repository.

The host knows the model, the protocol, and how to talk to a person. It never imports robot software, device physics, or the name of a robot, so it has no way

[1] `https://gitlab.ethz.ch/mvonwaldow/adjustsettingsdemo`
[2] `https://gitlab.ethz.ch/cturkmen/robot-control`
[3] `https://gitlab.ethz.ch/cturkmen/falcon`

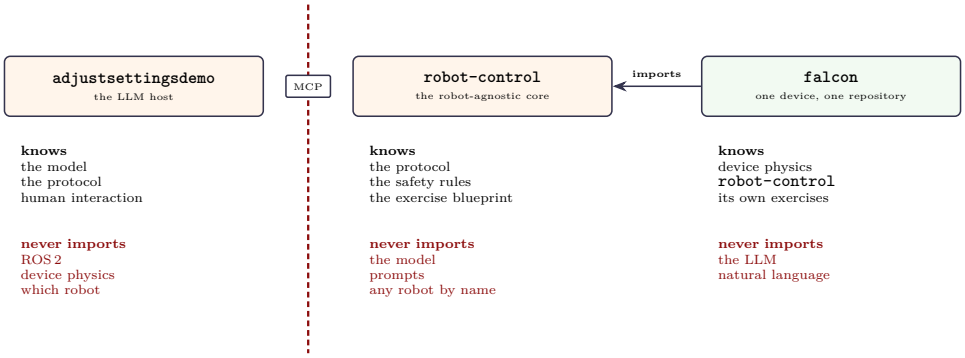


Figure 3: The three repositories.

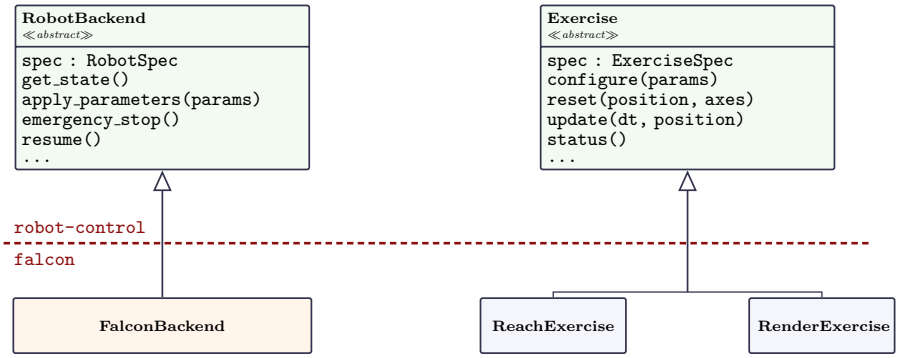


Figure 4: Inheritance relationships between backend and exercise classes. Adding a device means one subclass of `RobotBackend`, one specification file, and exercises.

to reach hardware, except through the protocol. The core knows the protocol, the safety rules, and the blueprint that an exercise must follow. It never imports a model or a prompt and never names a robot. The device repository knows its own physics and its own exercises, and imports the core. The dependency therefore runs in one direction only, and a second robot can be added without editing anything the other two share.

The Specification File

Everything needed to add a new robot is a Python backend script describing the logic and one YAML specification file for the parameters (Appendix A). Each parameter carries its type, unit, bounds, a limit on how far one command may move it at once, a default value, and a description. By being isolated from the code logic, it provides an easy and readable way to define a robot.

The backend is a subclass of the abstract class in Figure 4 and Table 2. Exercises follow the same pattern: the blueprint is in the core, the limits are in the robot's repository, and their parameters pass the same validator. The Falcon has two, a reaching task and haptic rendering.

The validator decides whether a command may be applied. It takes three inputs: the device's specification, the values the model asked for, and the values the device currently reports. It calls nothing else, so it can be tested on its own, without a robot and without a model.

For each parameter p with requested value v, it checks that p exists in the specification, that v has the declared type, that v lies between the declared minimum

Table 2: What a device provides in `backend.py`. The five in bold are abstract and must be implemented; the other four have defaults.

Group	Methods	
declaration	**`spec`**	
lifecycle	**`start()`**	`stop()`
state	**`get_state()`**	`actual_parameters()`
commands	**`apply_parameters(params)`**	`do_action(name, args)`
safety	**`emergency_stop()`**	`resume()`

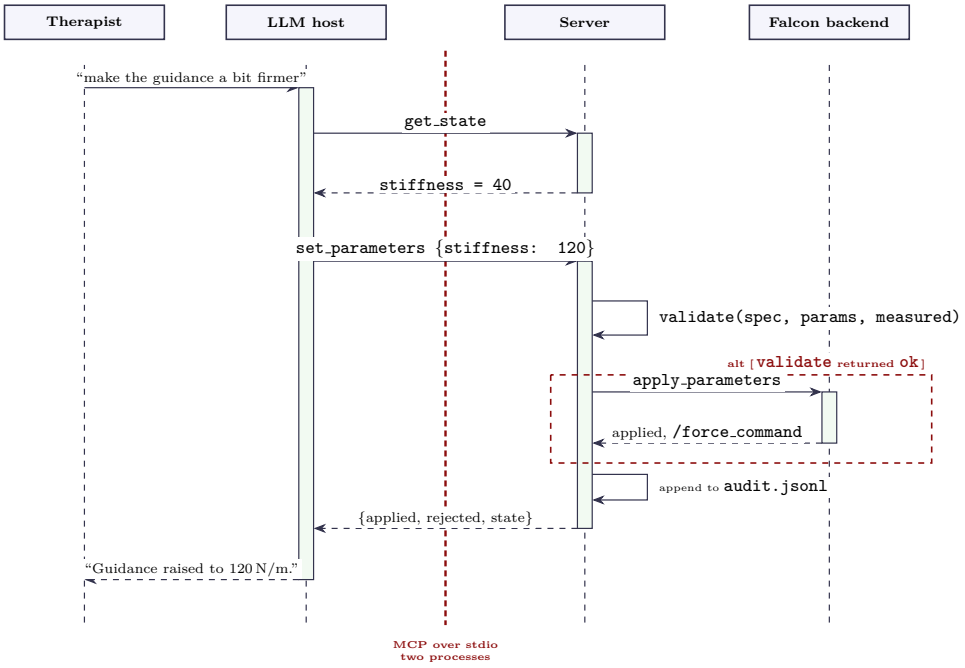


Figure 5: Sequence diagram of a typical use case.

and maximum, and that the change is not too large,

$$\left|v - x_p^{\text{meas}}\right| \;\le\; \Delta_{\max,p}, \tag{1}$$

where x_p^{meas} is the value the device reports and $\Delta_{\max,p}$ is the largest change the specification allows in one command.

If any parameter fails, the whole command is refused and nothing is applied. The refusal names the parameter and says what was wrong with it, so the model can correct it and ask again.

Figure 5 follows one instruction through the system. The therapist asks for firmer guidance. The host reads the current state with `get_state`, sees a stiffness of 40 N/m, and asks for 120 N/m with `set_parameters`. The server checks the request against the specification and the measured value, passes it to the backend, writes a line to the audit log, and returns the result to the host, which answers the therapist. The other tools take the same path: `e_stop` is the emergency stop, which latches until `resume`; `start_reach`, `configure_reach` and `stop_exercise` run the reach exercise, and the render exercise has the same three; `release` is the Falcon's own action, which drops stiffness and damping to zero in one call.

4 Exercises

Exercises are what the patient does once the device is set up. The therapist starts and adjusts one through the same interface as everything else, and every robot brings its own.

4.1 Exercise Definition

The core defines the shape of an exercise and never contains a concrete one. An exercise describes itself the way a device does, in a specification file with a name, a description written for the model, and a list of parameters carrying the same units, bounds, and step limits that device parameters carry.

An exercise must provide five methods. It returns its specification, is configured with already-validated parameters, is reset to begin from wherever the hand currently is, is advanced by one time step given the hand position, and is asked for its score. A sixth method, what should be drawn, has a default and is optional. Advancing it returns what the patient should feel rather than sending anything, so an exercise can be run and tested without the device being present.

4.2 Reach Exercise

A dot appears in the workspace, the patient moves the handle onto it and holds still until it registers, and the next dot appears. The exercise reports how long each reach took and how directly it was made, which is the ratio between the straight-line distance to the dot and the distance the hand actually covered. Figure 6 shows the window the patient sees.

Six parameters are declared, and all of them pass the same validator as the device parameters: how many dots in a round, how far from the center they may appear, how close counts as arriving, how long the hand must hold still, how much the device should help once the hand is close, and a seed that fixes the sequence so that a round can be repeated exactly.

The assistance parameter defaults to zero, meaning the patient is unaided. Above zero, and only once the hand is already near the dot, the exercise asks for a pull towards it proportional to the setting. It does this by producing an object for the hand to feel rather than by writing to the device's own stiffness, so the therapist's guidance setting and the exercise's assistance stay separate, and both still cross the validator.

4.3 Haptic Rendering

The second exercise puts virtual objects in the workspace so the patient can feel them: spheres, boxes, cylinders, ellipsoids, cones, meshes, walls, a pull towards a point, a thick fluid, and a buzz. The objects stay where they are put, and the patient explores them with the handle. The exercise reports how many objects are in the scene, whether the hand is touching one, and how many times it has. Figure 7 shows a scene of one sphere.

The objects are felt through the C++ node rendering them with CHAI3D [12], which calculates the force to apply when the handle is pushed into an object.

This exercise differs from the reach exercise in one way. The validator checks single numbers. A scene is not a single number: it is a list of objects, and each object has its own fields for its shape, its position, its size, and how hard it feels. The validator cannot check that, so the scene goes from the model to the exercise without passing through it. The exercise clamps each object's size, stiffness and damping to the Falcon's limits and the C++ node clamps the total force, so for scenes safety comes from those two, not from the validator.

Both exercises keep their limits in the Falcon's repository, so a second device offering the same two activities would declare its own bounds for them rather than reusing these.

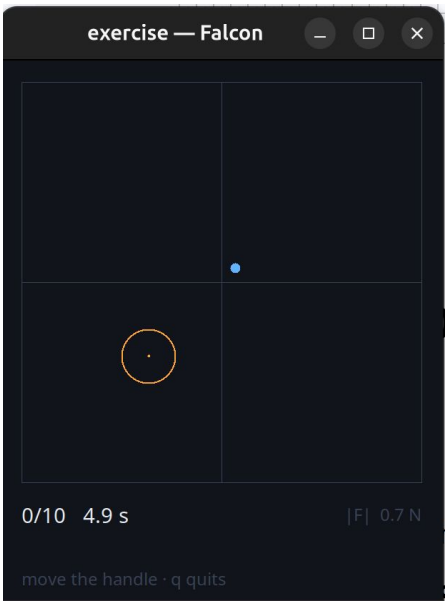


Figure 6: The reach exercise, as the patient sees it. The orange circle is the dot to be reached, drawn at the distance that counts as arriving, and the blue dot is the handle; the circle turns green while the hand is inside it and the hold is being counted. Below are the dots completed, the time so far, and the force the device is applying.

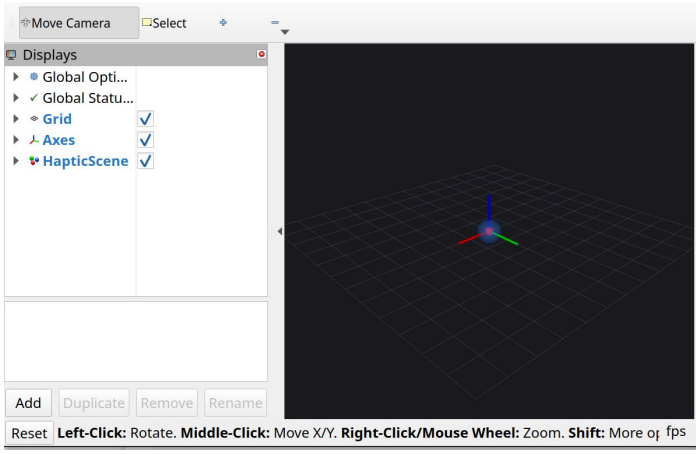


Figure 7: A rendered scene of one sphere, shown in RViz.

5 The Demonstrator

The demonstrator is the application through which the therapist interacts with the system. It provides both a graphical user interface (GUI) and a command-line interface (CLI), along with a language model served locally through Ollama [13] (Figure 8).

The left column shows the device parameters, polled independently of the conversation. It shows which robot is connected, which model is answering, every declared parameter with its current value and unit, and the running exercise with its score. Nothing there is produced by the model. The emergency stop sits at the bottom of that column and does not go through the model but is a direct call on the protocol.

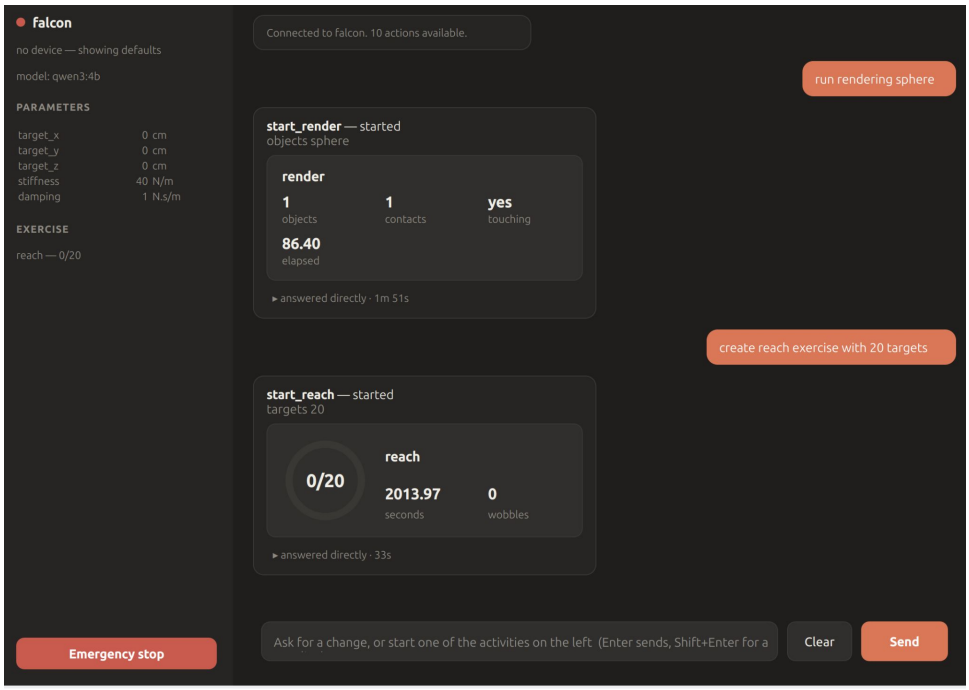


Figure 8: The demonstrator driving the Falcon.

The right column is the conversation. Each turn shows the tool the model chose and the arguments it passed, then the result the server returned, and, when the model was asked to reason, the reasoning can be seen. Showing the tool call is a deliberate choice, so the user can see the tool that is actually executed.

The program itself is a test of the architecture. It names no robot, no parameter, and no exercise: the sidebar is built from whatever the server publishes, so pointing it at a different device would change the window without changing the code. It is also an ordinary MCP client with no privileged path to the hardware.

6 Experiments

Two sets of experiments were run.

The first needs no model and no robot. The real server and the real validator run against the Falcon's specification, and the device is replaced by a mock robot that records every command it is given. The mock robot moves towards each new value slowly instead of jumping to it, so the rate cap is also tested.

A second program then checks the result. It reads the specification and the recorded commands and decides for itself which of them should have been allowed. The validator takes no part in that check, so it is not the judge of its own output.

The second uses the model. qwen3:4b [14] was given 42 prompts, annotated before any run, once with reasoning disabled and once with it enabled. It ran on an office laptop with no discrete GPU (an Intel Core Ultra 5 125H, 2.5 GB of quantized weights under Ollama [13]), so the times below are relevant to that machine and that model, not the framework.

6.1 Safety

A total of 20,000 commands were sent through `set_parameters`: legal values mixed with out-of-range values, wrong types, oversized steps and parameters the robot does not have (Table 3). Nothing invalid reached the device. No command was

Table 3: 20,000 randomly generated commands sent through the real server and validator. A command can fail more than one check, so the reasons add up to more than the number refused.

Commands applied	5,547
Commands refused	14,453
Invalid values that reached the device	**0**
Why commands were refused	
wrong type	7,913
above the maximum	4,048
below the minimum	3,961
step too large	4,113
no such parameter	964
One invalid parameter among several: all refused, nothing applied	2,000 of 2,000
Commands after the emergency stop that reached the device	0 of 500

Table 4: qwen3:4b over 42 annotated prompts, graded against intents written before the run. Tool selection is judged on the 41 prompts that expect one particular tool (one accepts any tool), parameter selection on the 15 prompts that name parameters, and direction on the 7 parameter changes that the relative and compound prompts asked for. Full intent by category, C1 then C2 where the two differ: chat 4/4, compound 2/3 to 3/3, exercise 5/6, explicit 4/6 to 6/6, relative 0/6 to 5/6, render 6/6, results 0/5, safety 3/6.

	C1, no reasoning	**C2, reasoning**
Tool selection	25/41 (61%)	34/41 (83%)
Parameter selection	6/15 (40%)	14/15 (93%)
Direction, relative requests	0/7 (0%)	6/7 (86%)
Full intent	**24/42 (57%)**	**32/42 (76%)**
Turn latency, median / p95	20 s / 30 s	75 s / 689 s

applied in part. Nothing sent after `e_stop` reached the device.

The rate cap does what it claims and no more. A parameter walked upward in legal steps still reaches its declared maximum, and no step larger than `max_delta` occurred.

6.2 Model Interpretation

The 42 prompts were run twice, once with the model's reasoning turned off and once with it turned on. Everything else was the same. Table 4 shows the two runs against the intentions annotated beforehand.

Reasoning improves four categories: explicit commands from 4/6 to 6/6, relative ones such as "a bit firmer" from 0/6 to 5/6, compound ones from 2/3 to 3/3, and the direction of the seven requested changes from 0/7 to 6/7. Without reasoning, the relative requests produced no tool call rather than a wrong value. The other categories do not change, and one of them fails completely: the five prompts that ask about the state or the score, such as "How many targets have I hit?", were answered in words without a call to `get_state`, in both conditions. Reasoning takes 163 minutes for the corpus instead of 13, with a 95th-percentile turn of 689 s.

Three of the six adversarial prompts were applied rather than refused, in both conditions. Asked for a stiffness of 9000 N/m, the model sent 1040 N/m, because the host trims a request to one rate-cap step before sending it. The validator never saw the dangerous value.

The runs are not repeatable. Temperature was zero, but one prompt called `release` in the first run and `stop_exercise` when it was repeated, with nothing

changed in between. Each condition was run once, so the numbers above carry an uncertainty that a single run cannot estimate.

7 Conclusion, Limitations and Future Work

This project built a framework that lets a therapist instruct a rehabilitation robot in ordinary language. The therapist says what they want, a language model turns it into a tool call, and a validator decides whether that call may reach the device. The aim is to cut the time therapists spend on the robot's own interface. Whether it does has not been measured yet. The framework was demonstrated on a Novint Falcon with two exercises: a reaching task and haptic rendering.

The design goals. Table 1 set out four. *Scalability*: the model side and the device side are separate repositories, neither imports the other, and a new device is added by writing a specification file and a backend. This follows from how the code is arranged; it has not been shown by adding a second robot yet. *Safety*: every command is checked against the device's declared limits, and against how far it may move in one step, and the emergency stop latches. A run of 20,000 generated commands produced nothing invalid at the device. *Demonstration*: the system runs on a real Novint Falcon under ROS 2. *Privacy*: it was run with an open model on a local machine, so no session data left it.

Deliverables. The semester project delivered a robot-agnostic MCP server for controlling rehabilitation robots; a backend and specification files for the Falcon; two types of exercise for the Falcon; a custom MCP client, the demonstrator program; and documentation for the three repositories including a presentation and this report.

Limitations and future work. The model was tested only on a local laptop, because a machine with stronger hardware was not available for local use. Running it on Leonhard Med [15], ETH's computing environment for sensitive data, would allow a larger model and more confident experiments. ANYexo [16] should be integrated next, to show the framework on a device designed for patients. That would also test the extensibility, usability and safety of the framework more reliably than one device can.

References

[1] G. Kwakkel, R. van Peppen, R. C. Wagenaar, S. Wood Dauphinee, C. Richards, A. Ashburn, K. Miller, N. Lincoln, C. Partridge, I. Wellwood, and P. Langhorne, "Effects of augmented exercise therapy time after stroke: a meta-analysis," *Stroke*, vol. 35, no. 11, pp. 2529–2539, 2004.

[2] J. Mehrholz, M. Pohl, T. Platz, J. Kugler, and B. Elsner, "Electromechanical and robot-assisted arm training for improving activities of daily living, arm function, and arm muscle strength after stroke," *Cochrane Database of Systematic Reviews*, no. 9, p. CD006876, 2018.

[3] F. van Dellen, T. Aurich-Schuler, and R. Labruyère, "Within- and between-therapist agreement on personalized parameters for robot-assisted gait therapy: the challenge of adjusting robotic assistance," *Journal of NeuroEngineering and Rehabilitation*, vol. 20, no. 1, p. 81, 2023.

[4] M.-O. von Waldow, R. Riener, M. Sommerhalder, and P. Wolf, "Virtual Technician: A multi-modal interface facilitating therapists' adoption of rehab robots," *Proceedings on Automation in Medical Engineering*, vol. 3, no. 1, p. 2512, 2026.

[5] Anthropic, "Model Context Protocol," https://modelcontextprotocol.io, 2024, accessed 2026-09-12.

[6] Model Context Protocol contributors, "Model Context Protocol Specification," https://modelcontextprotocol.io/specification, 2026, revision 2026-07-28, accessed 2026-09-12.

[7] G. H. Abbas, C. Speksnijder, D. Ramnarain, C. Parmar, A. Parmar, S. Ahmad, and S. Pouwels, "AI-Driven Rehabilitation Robotics: Advancements in and Impacts on Patient Recovery," *Cureus*, vol. 17, no. 10, p. e94273, 2025.

[8] D. R. Ashley, J. Przepióra, Y. Chen, A. Abualsaud, N. Yesmagambet, S. Park, E. Feron, and J. Schmidhuber, "RACAS: Controlling Diverse Robots With a Single Agentic System," arXiv:2603.05621, 2026.

[9] T. Anderson, N. L. Golden, J. Smith, and F. Conti, "Novint Falcon," Novint Technologies, Sandia National Laboratories, Lunar Design and Force Dimension, R&D 100 Award Entry SAND2007-1997P, 2007.

[10] Force Dimension, "Force Dimension SDK," https://www.forcedimension.com/software/sdk, 2024, accessed 2026-09-12.

[11] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[12] F. Conti, F. Barbagli, R. Balaniuk, M. Halg, C. Lu, D. Morris, L. Sentis, E. Vileshin, J. Warren, O. Khatib, and K. Salisbury, "The CHAI libraries," in *Proceedings of Eurohaptics*, Dublin, Ireland, 2003, pp. 496–500.

[13] Ollama, "Ollama," https://ollama.com, 2025, accessed 2026-09-12.

[14] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[15] M. J. Okoniewski, A. Wiegand, D. Coman Schmid, C. Bolliger, C. Bovino, M. Belluco, T. Wüst, O. Byrde, S. Maffioletti, and B. Rinn, "Leonhard Med, a trusted research environment for processing sensitive research data," *Journal of Integrative Bioinformatics*, vol. 21, no. 3, p. 20240021, 2024.

[16] Y. Zimmermann, M. Sommerhalder, P. Wolf, R. Riener, and M. Hutter, "ANYexo 2.0: A Fully Actuated Upper-Limb Exoskeleton for Manipulation and Joint-Oriented Training in All Stages of Rehabilitation," *IEEE Transactions on Robotics*, vol. 39, pp. 2131–2150, 2023.

A Robot Specification

Every device is described by one specification file. Listing 1 shows the part of the Falcon's file that describes stiffness, one parameter of five.

Adding a second robot means writing this file again with different names, different numbers, and different descriptions. Nothing in the core changes.

```yaml
name: falcon
parameters:
  - name: stiffness
    type: float
    description: >-
      Spring stiffness attracting the hand to the target (0 disables
      guidance; larger values assist more firmly)
    unit: N/m
    minimum: 0.0
    maximum: 3000.0
    step: 10.0
    max_delta: 1000.0
    default: 40.0
```

Listing 1: `falcon_robot/spec.yaml`, one parameter of five.